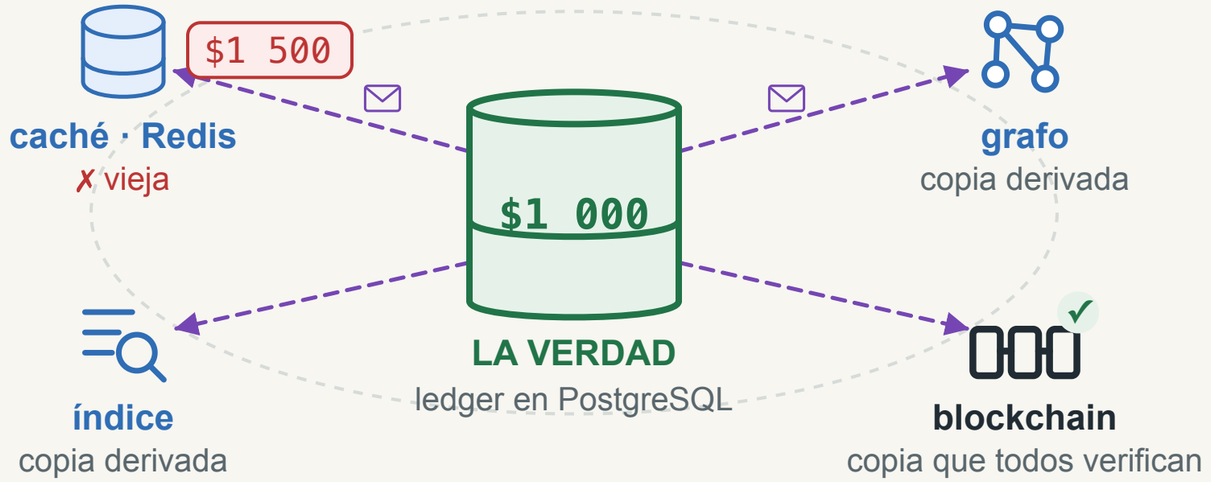
Una verdad, varias copias

Bases de datos, caché y blockchain

LA PREGUNTA DE HOY

¿Dónde vive cada dato,
y qué pasa cuando
vive en más de
un lugar?

MEXI BANCO / LA VERDAD Y SUS COPIAS



Políglota | SQL contra NoSQL | Las familias | Uno contra otro | Caché | Blockchain

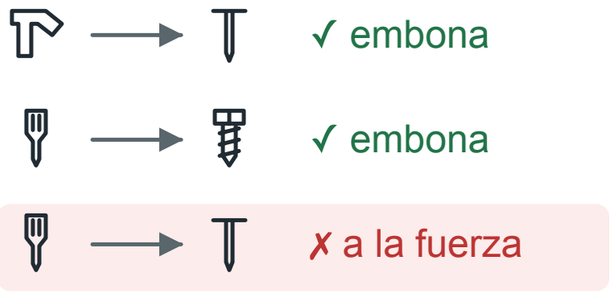
Cada dato tiene una verdad y, casi siempre, varias copias.

La herramienta correcta

¿Dónde vive cada dato?

Persistencia políglota: cada dato vive en el almacén que mejor sirve a cómo se usa.

1 No clavas con desarmador



2 El martillo relacional

A un equipo le iba mejor una clave-valor que

«el martillo relacional corporativo».

3 Programación políglota

`2006` **Neal Ford**
cada trabajo, su lenguaje

`2011` **Fowler**
cada dato, su almacén

EN MEXI BANCO

Ledger → **PostgreSQL** ✓ transacción local

Saldo para mostrar → **Redis** copia que puede ir atrás

Primero cómo se usa el dato; después, el motor.

Ninguna base gana en todo

¿Por qué no hay una base que sea la mejor para todo?

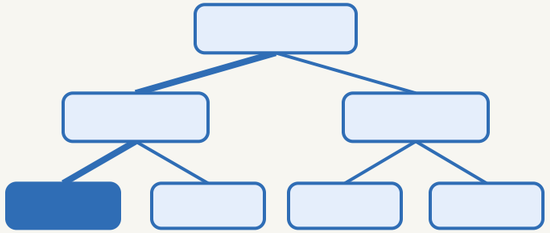
Cada motor ya decidió cómo se escribe, cómo se lee y qué se garantiza.

1. **¿Cómo se escribe?**
2. **¿Cómo se lee?**
3. **¿Qué se garantiza?**

EL DILEMA MÁS CHICO: UN ÍNDICE



lee fila por fila



copia ordenada de las CLABE

	SIN ÍNDICE	CON ÍNDICE
Leer una CLABE de 1 000 000	✗ 1 000 000 filas	✓ unos pocos nodos
Insertar una cuenta	✓ 1 escritura	✗ tabla + índice

Cada estructura que acelera una lectura encarece una escritura.

El e-commerce de Fowler

¿Cómo se ve una aplicación con un almacén por necesidad?

Cada dato se lee, se escribe y se cuida distinto, y por eso cambia de almacén.

Sesiones de usuario



Redis

efímero

Datos financieros



RDBMS

registro

Carrito de compras



Riak

efímero

Recomendaciones



Neo4J

derivado

Catálogo de productos



MongoDB

registro

Reportes



RDBMS

derivado

Analítica



Cassandra

derivado

Bitácora de actividad



Cassandra

registro

⚠

«No tomen demasiado en serio el ejemplo del diagrama.»

registro es la verdad

derivado sale de otro

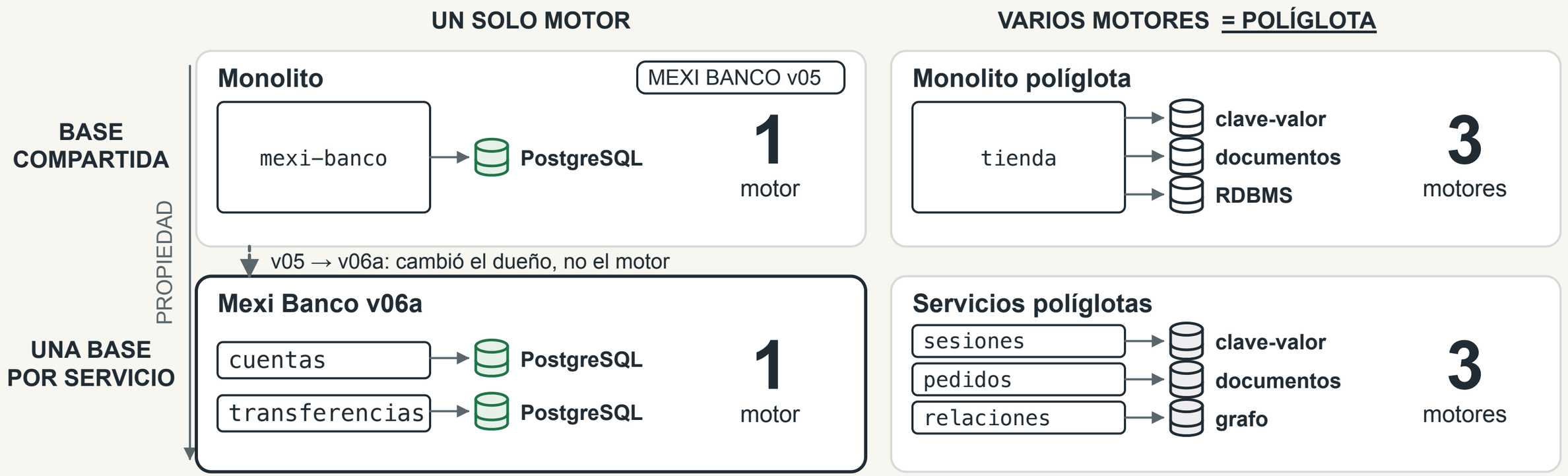
efímero se puede perder

La lección es la pregunta por cada dato, no la marca de cada motor.

Políglota no es «una base por servicio»

¿Tener una base por servicio ya es ser políglota?

Una base por servicio habla de propiedad; políglota, de cuántos motores distintos hay.



Quién es dueño y qué motor usa son dos decisiones distintas.

Lo que cuesta

¿Qué pagas por cada motor que agregas?

Aquí se rompe la caja de herramientas: las herramientas no comparten estado y las bases sí.

Herramientas



✓ nada que sincronizar

Bases



1 Transacción



`@Transactional`

PostgreSQL

Redis ✗ fuera

Se cubre con saga (S08) y con Outbox (S09).

2 Disponibilidad



$0.999^4 \approx 99.6\ \%$

1 almacén	≈ 43 min
4 almacenes	≈ 2.9 h

caída al mes si se usan los 4

3 Antipatrones



✗ **Políglota prematuro**, antes de conocer el acceso

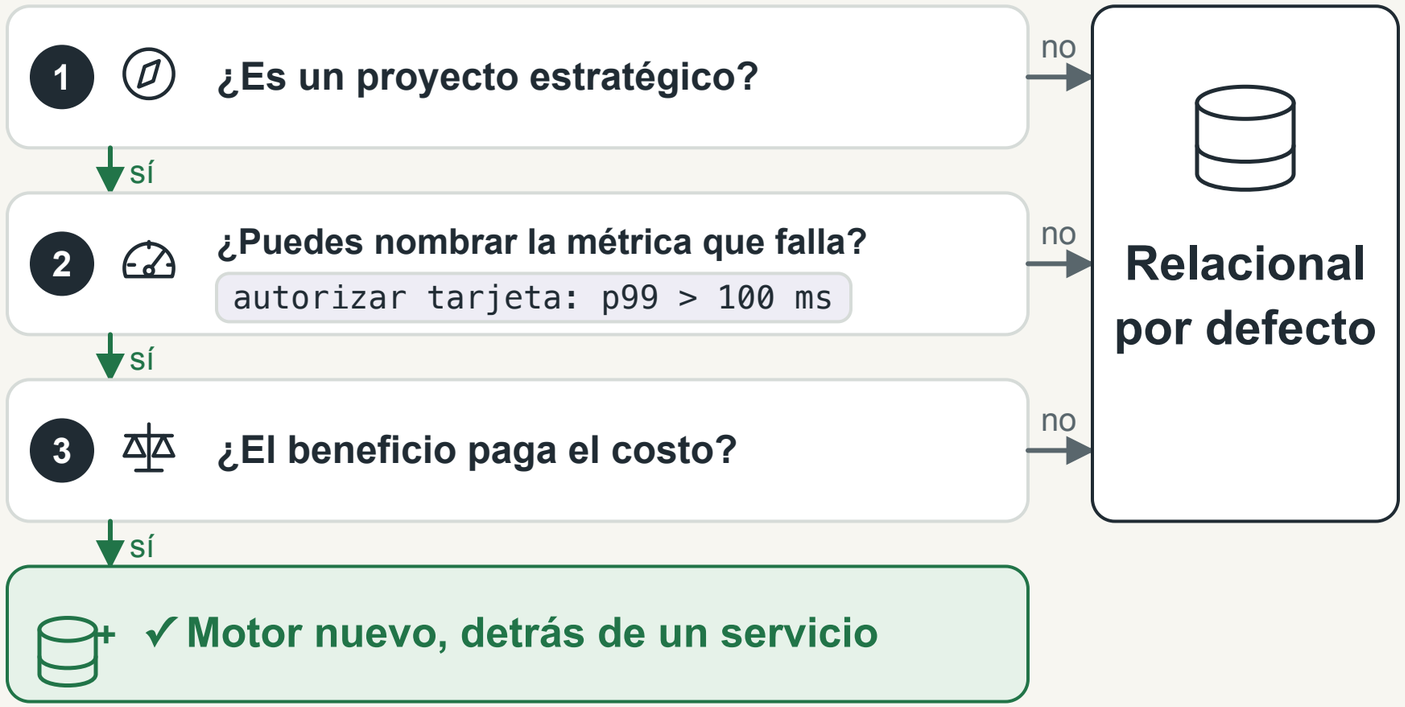
✗ **Un motor nuevo** por una mejora menor

Una herramienta más cuesta una vez; una base más cuesta todos los días.

Cuándo sí

¿Cuándo sí vale la pena un motor más?

Fowler: «probablemente la mayoría está mejor con la ortodoxia relacional».



SEÑALES DE QUE SÍ

Modelos de datos muy distintos

Rendimiento: órdenes de magnitud

< 100 ms contra 24 h, mismo banco

Equipo que ya opera varios motores

Relacional por defecto. ¿Qué significa de verdad SQL contra NoSQL?

Modelo y patrón de acceso

Entonces, ¿qué significa de verdad SQL contra NoSQL?

No es el lenguaje ni la velocidad: es el modelo de datos y si ya conoces tus consultas.

1 Relacional

Guarda separado y reúne al leer



✓ Perdona no conocer tus consultas

Costo: reunir con joins en cada lectura

2 Agregados (NoSQL)

Guarda junto lo que se lee junto

```
{ "cliente": "Ana",
  "clabe": "012180015678912345",
  "movs": [500, −120] }
```

una lectura, por cliente

✓ Rápido y barato para lo que planeaste

✗ Fuera de eso: caro y lento

La pregunta no es SQL o NoSQL: es si ya conoces tus consultas.

Esquema al escribir o al leer

¿De verdad NoSQL no tiene esquema?

Esquema al escribir: lo revisa la base. Esquema al leer: lo revisa tu código.

LA MISMA TAREA: AGREGAR EL RFC DEL TITULAR

1 Al escribir

PostgreSQL

Como el tipado estático

```
ALTER TABLE cuenta
ADD COLUMN rfc_titular VARCHAR(13);
```

```
INSERT INTO cuenta (clabe, titular, rfc_titular)
VALUES ('014180655091234567', 'Ana',
        'GOMA800101AB12');
```

```
✗ ERROR: value too long for type
  character varying(13)
```

✓ La base rechaza el RFC de 14 caracteres

Costo: cada cambio de forma es una migración

2 Al leer

MongoDB

Como el tipado dinámico

```
db.cuentas.insertOne({
  _id: "014180655091234560", titular: "Luis" })
db.cuentas.insertOne({
  _id: "014180655091234567", titular: "Ana",
  rfc_titular: "GOMA800101AB12" })
```

```
String rfc = doc.getString("rfc_titular");
// Luis: null
if (rfc == null) rfc = "SIN RFC";
```

✓ Conviven las dos formas, sin migrar

✗ Nadie rechazó el RFC de 14 caracteres

El esquema siempre existe: lo revisa la base o lo revisa tu código.

Normalizar contra desnormalizar

¿Guardo cada dato una vez o lo copio donde se lee?

Normalizar: cada dato vive en un lugar y se reúne con joins al leer.

DESNORMALIZADO

PedidoID	Cliente	EmailCliente	TotalPedidos
1001	Alice	aliceNEW@ej.com	1500
1002	Bob	bob@ej.com	800
1003	Alice	alice@ej.com	300

✗ Cambió el correo y solo se tocó una fila

NORMALIZADO

ClienteID	Cliente	Email
C1	Alice	aliceNEW@ej.com
C2	Bob	bob@ej.com

PedidoID	ClienteID	Total
1001	C1	1500
1002	C2	800
1003	C1	300

✓ Una sola copia

SI ALICE TIENE 10,000 MOVIMIENTOS

Correo copiado en cada movimiento — **10,000 escrituras**

Correo en cliente — **1 escritura**

Diseñar por las consultas

Copia solo si conoces la consulta y el dato casi no cambia.

Mexi Banco: estado de cuenta, por lotes

Normalizar paga al leer; desnormalizar paga al escribir.

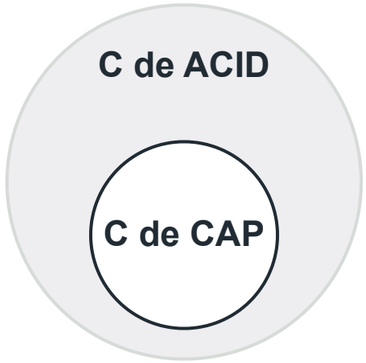
ACID contra BASE

¿Son dos bandos o dos extremos de lo mismo?

ACID: todo o nada, y la base queda válida. BASE: responde siempre y converge después.

ACID — ✓ Cargo y abono en cuentas — Saga entre servicios — Cajero sin red — Saldo que se muestra — **BASE**

1 Las dos C



ACID: respeta las reglas
`UNIQUE, saldo ≥ 0`

CAP: una sola copia

2 El cajero de Brewer

Invariante
`saldo ≥ 0`

Sin red
retiro neto ≤ k
k = $200

Al volver
compensa: comisión y cobro

Mexi Banco: la autorización de tarjeta (< 100 ms)

ACID y BASE son extremos de un espectro, y un banco usa los dos.

El mapa

Si NoSQL no es una sola cosa, ¿qué familias hay?

Una familia es una pregunta típica más la estructura de datos que la vuelve barata.

1 Relacional

¿Se cumplen las reglas?

2 Clave-valor

¿Qué hay en esta llave?

3 Documentos

¿Me das el expediente completo?

4 Columnas anchas

¿Esta partición, en orden?

5 Grafos

¿Cómo se conecta esto con aquello?

6 Búsqueda

¿Qué documentos dicen esto?

7 Series de tiempo

¿Cómo se comportó esta métrica?

8 Vectorial

¿Qué se parece a esto?

Puente · NewSQL la pregunta de la relacional, cuando un nodo no alcanza

Pregunta y estructura primero; el nombre del motor viene después.

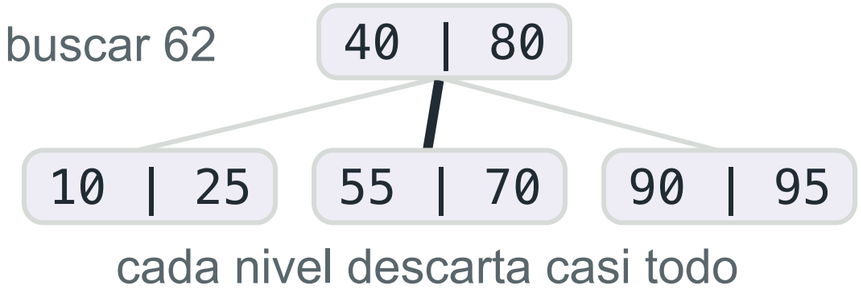
Relacional

¿Qué es verdad ahora y se cumplen las reglas?

Tablas con esquema, llaves y restricciones; se consulta con SQL y cada transacción es ACID.

ESTRUCTURA QUE LA ABARATA

B-tree, WAL y MVCC



BRILLA EN

- ✓ Reglas entre filas
- ✓ Escrituras todo o nada
- ✓ Consultas no previstas

NO BRILLA EN

- ✗ Escribir más que un nodo
- ✗ Esquemas muy variables
- ✗ Texto con relevancia

CASO · **Stack Overflow**

504,816,843 consultas SQL en un día con 2 clústeres

EN MEXI BANCO — ✓ VERDAD

cuentas: `CHECK (saldo >= 0)`

Donde hay reglas entre filas, la verdad vive en la relacional.

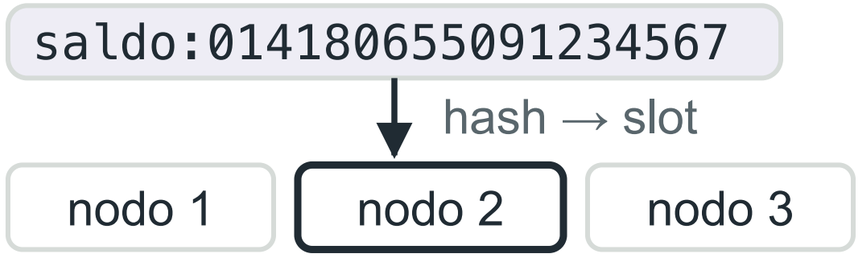
Clave-valor

¿Qué hay en esta llave? Lo quiero ya.

Un diccionario distribuido: la llave es opaca para el motor.

ESTRUCTURA QUE LA ABARATA

Tabla hash particionada



BRILLA EN

- ✓ Latencia mínima
- ✓ Escala casi lineal
- ✓ Contadores, TTL, rankings

NO BRILLA EN

- ✗ Buscar por otro campo
- ✗ Reglas entre llaves
- ✗ Ser la verdad (Redis)

CASO · **Tabla de posiciones en vivo**

`ZINCRBY` y `ZREVRANGE`: top N en **O(log n)**

EN MEXI BANCO — **COPIA DERIVADA**

saldo para mostrar

Por llave, nadie le gana; para todo lo demás, está ciega.

Documentos

¿Me das el expediente completo en una sola lectura?

Colecciones de documentos JSON: cada uno es un agregado con datos anidados.

ESTRUCTURA QUE LA ABARATA

Agregado anidado (JSON)



BRILLA EN

- ✓ Leer el agregado entero
- ✓ Atributos que varían
- ✓ Escribir un documento

NO BRILLA EN

- ✗ Muchos a muchos
- ✗ Reglas entre documentos
- ✗ Agregados sin tope

CASO · **Un menú, un documento**

En producción: eBay, el carrito de cada usuario es **un solo JSON**.

EN MEXI BANCO ✓ VERDAD

notificaciones: preferencias de C-1027

Lo que se lee junto, se guarda junto en un documento.

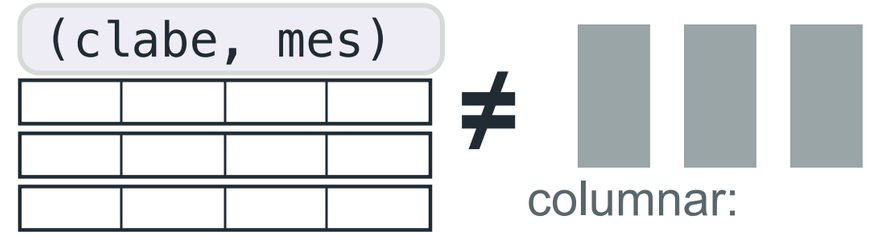
Columnas anchas

¿Cómo leo esta partición en orden mientras entran millones de escrituras?

Una partition key agrupa filas ordenadas por una clustering key; se consulta con CQL.

ESTRUCTURA QUE LA ABARATA

Partición ordenada sobre LSM-tree



✓ guarda por fila

BRILLA EN

- ✓ Escrituras masivas
- ✓ Rangos en una partición
- ✓ Crecer agregando nodos

NO BRILLA EN

- ✗ Consultas no previstas
- ✗ Sumar la tabla entera
- ✗ Borrar mucho

CASO · **Discord**

177 nodos de Cassandra → **72** de ScyllaDB: misma familia

EN MEXI BANCO — **COPIA DERIVADA**

historial por cuenta, a escala

Columnas anchas guarda por fila dentro de cada partición: no es columnar.

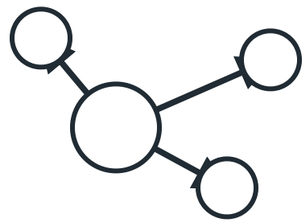
Grafos

¿Cómo se conecta esto con aquello, y a cuántos saltos?

La relación se guarda como dato; no se recalcula con un join.

ESTRUCTURA QUE LA ABARATA

Adyacencia por punteros



✓ salto = puntero



salto = un join

BRILLA EN

- ✓ Profundidad variable
- ✓ Patrones con forma
- ✓ Relaciones nuevas

NO BRILLA EN

- ✗ Repartirse en shards
- ✗ Reportes y sumas masivas
- ✗ Profundidad fija

CASO · **LinkedIn Llquid**

≈ 250 búsquedas aleatorias para el 2.º grado con un join

EN MEXI BANCO — **COPIA DERIVADA**

antifraude: red de transferencias

Grafos cuando la profundidad es variable, no solo porque haya relaciones.

Búsqueda

¿Qué documentos dicen esto, ordenados por relevancia?

El índice invertido guarda, para cada término, la lista de documentos que lo dicen.

ESTRUCTURA QUE LA ABARATA

Índice invertido

«Pago renta octubre depto Coyoacán»

↓ minúsculas y sin acentos

```
renta → MOV-88213, MOV-90410
coyoacan → MOV-88213
```

BRILLA EN

- ✓ Texto libre
- ✓ Relevancia y facetas
- ✓ Escalar lecturas

NO BRILLA EN

- ✗ Ser la verdad
- ✗ Leer lo recién escrito
- ✗ Reescribir lo mismo

CASO · **«renta coyoacan» contra LIKE**

✗ `LIKE '%renta%'` barre la tabla · ✓ el índice va directo a **2 términos**

EN MEXI BANCO — **COPIA DERIVADA**

movimientos: índice de búsqueda

LIKE barre la tabla; el índice invertido va directo a los términos.

Series de tiempo

¿Cómo se comportó esta métrica en esta ventana?

(métrica, etiquetas, instante) → valor; los datos solo se agregan al final.

ESTRUCTURA QUE LA ABARATA

Bloques por tiempo

instante (s)	**0**	15	30	46	61
delta		15	15	16	15
delta de delta		**0**	+1	−1	

`0 → 1 bit`

BRILLA EN

- ✓ Ingesta masiva
- ✓ Mucho en poco espacio
- ✓ Retención barata

NO BRILLA EN

- ✗ Corregir puntos sueltos
- ✗ Alta cardinalidad
- ✗ Registros de negocio

CASO · **Facebook Gorilla**

de 16 a 1.37 bytes por punto: **12 veces** menos espacio

EN MEXI BANCO

métricas del SPEI

TELEMETRÍA

Series de tiempo para la telemetría que se resume; el dinero no se muestrea.

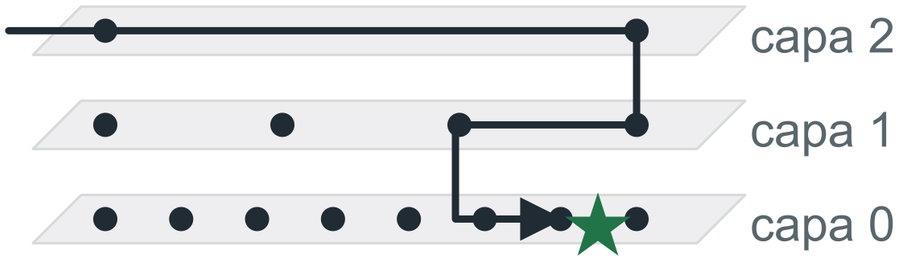
Vectorial

¿Qué se parece a esto?

Cada registro lleva un embedding: un vector de cientos o miles de números que codifica su significado.

ESTRUCTURA QUE LA ABARATA

Índice ANN: HNSW



BRILLA EN

- ✓ Parecido por significado
- ✓ Sin revisar todo
- ✓ Filtros híbridos

NO BRILLA EN

- ✗ Exactitud
- ✗ Memoria
- ✗ Ser la verdad

CASO · **Spotify**

de Annoy a Voyager (HNSW): **> 10×** más rápida con igual recall

EN MEXI BANCO — **COPIA DERIVADA**

soporte: tickets parecidos

Vectorial para lo que se parece; es un índice derivado, no la verdad.

NewSQL, el puente

¿Y si necesito la relacional, pero un nodo ya no alcanza?

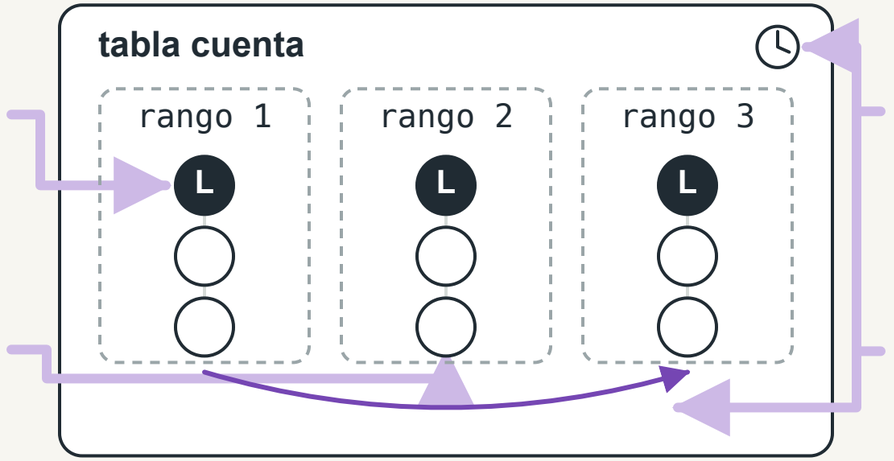
SQL y ACID como en la relacional, pero el motor parte cada tabla en rangos.

1 Raft S06

Cada rango elige su líder.

2 Sharding S07

La tabla se parte en rangos.



3 2PC S08

Cargo en 1, abono en 3.

4 Relojes S10

TrueTime: error < 10 ms.

⚖ LO QUE CUESTA

✗ Cada escritura: una ronda de consenso

✗ Entre regiones: decenas o cientos de ms

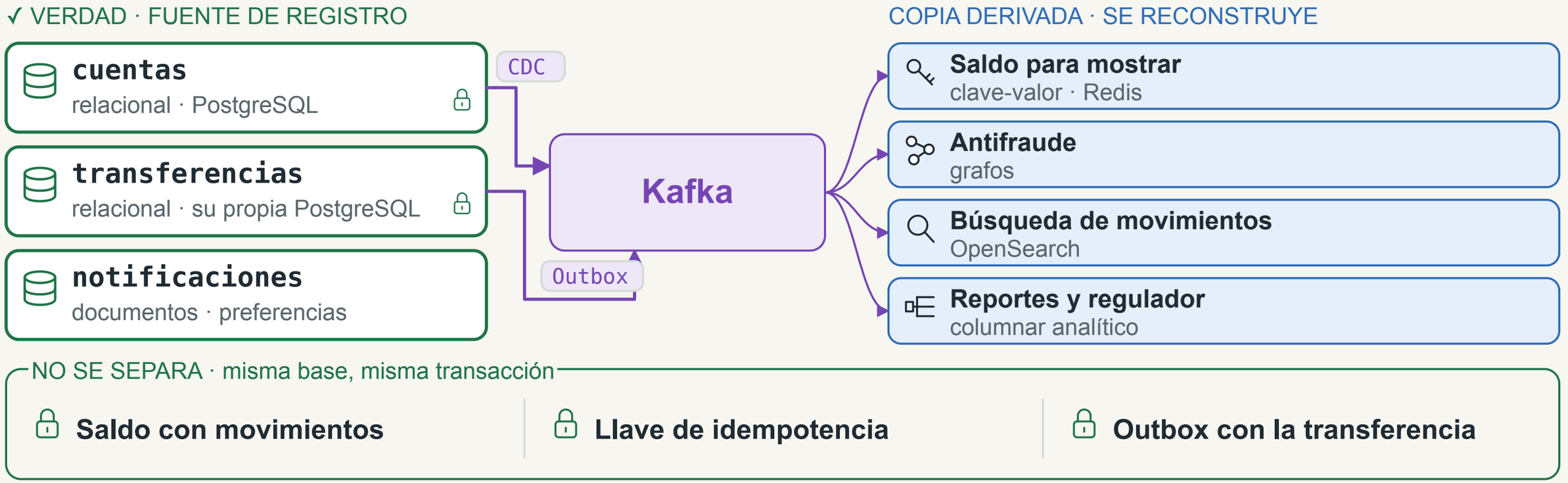
✗ Ids que solo crecen: un rango caliente

NewSQL no es otra familia: es la relacional repartida con todo el curso.

Mexi Banco políglota

¿Dónde vive cada dato, y cuál es la verdad?

La verdad está en un solo lugar; lo demás son copias derivadas que se pueden reconstruir.



La verdad, junta; lo demás, copias. Con un caso concreto, ¿cómo se decide?

El saldo

¿Relacional o documentos?

Invariante del saldo: nunca baja de cero, y cada transferencia mueve dos a la vez.

ESCENARIO | **Mexi Banco** · Ada López transfiere $9,500 a otra cuenta: cargo y abono, o nada.

✓ GANA

Relacional
PostgreSQL

`CHECK (saldo >= 0)`

una transacción

```
BEGIN;
UPDATE cuenta SET saldo = saldo - :m
 WHERE clabe = :o AND saldo >= :m;
UPDATE cuenta SET saldo = saldo + :m
 WHERE clabe = :d;
COMMIT;
```

✓ Dos filas, una transacción local

vs

Documentos
MongoDB

`{ _id: clabe, saldo }`

```
const s = db.getMongo().startSession();
const c = s.getDatabase("banco").cuentas;
s.startTransaction();
c.updateOne({ _id: o, saldo: { $gte: m } },
            { $inc: { saldo: -m } });
c.updateOne({ _id: d }, { $inc: { saldo: m } });
s.commitTransaction();
```

en cada transferencia

✗ Dos documentos: transacción en cada operación

Si cada operación toca dos documentos, tu modelo es relacional.

Anillos de fraude

¿Grafo o `WITH RECURSIVE`?

Anillo: el dinero sale de una cuenta y vuelve a ella en 3 a 6 saltos dentro de 24 h.

ESCENARIO | **Mexi Banco** · antifraude revisa la cuenta A

Relacional
PostgreSQL

✓ BASTA A 2 SALTOS

```
WITH RECURSIVE r(cta, n) AS (
  SELECT :clabe, 0
  UNION ALL
  SELECT t.destino, r.n + 1
  FROM r JOIN transferencia t ON t.origen = r.cta
  WHERE r.n < 6                    ↑ un join por salto, hasta 6
    AND t.ts > now() - interval '24 hours')
SELECT count(*) FROM r
WHERE cta = :clabe AND n >= 3;
```

✗ De 3 a 6 saltos: un join por salto

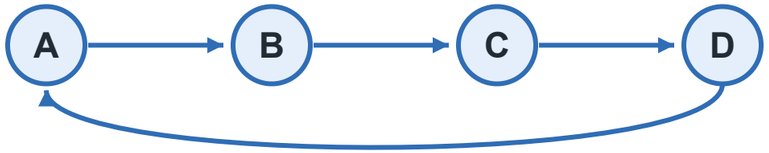
vs

Grafos
Neo4j

✓ GANA

copia derivada

```
MATCH ruta = (c:Cuenta {clabe: $clabe})
  -[:TRANSFIRIO*3..6]->(c)     3 a 6 saltos, de vuelta a c
WHERE all(t IN relationships(ruta)
  WHERE t.ts > datetime() - duration('P1D'))
RETURN count(ruta);
```



✓ Sigue punteros: cuesta lo que recorre

Grafo si la profundidad es variable; joins si es fija.

Boletos que expiran

¿El TTL de Redis o `UNIQUE (evento, asiento)`?

Apartar un asiento 10 minutos: si no pagas se libera, y nunca puede tener dos dueños.

ESCENARIO | **Venta de boletos** · dos personas eligen el F12 del evento 77 en el mismo segundo.

⚠ TIENTA

Clave-valor
Redis

```
> SET asiento:ev77:F12 C-1027 NX EX 600
OK    confirmado a C-1027
```

✗ Failover: la réplica asciende sin esa llave.

```
> SET asiento:ev77:F12 C-2048 NX EX 600
OK    ✗ también a C-2048: dos dueños
```

✗ Un failover puede borrar un OK confirmado

vs

✓ GANA

Relacional
PostgreSQL

`UNIQUE (evento, asiento)`

```
INSERT INTO reserva (evento, asiento, cliente,
  expira_en) VALUES (77, 'F12', 'C-2048',
  now() + interval '10 minutes')
ON CONFLICT (evento, asiento) DO UPDATE
  SET cliente = EXCLUDED.cliente,
      expira_en = EXCLUDED.expira_en
  WHERE reserva.expira_en < now();   solo si ya venció
```

✓ El índice UNIQUE deja un solo dueño

La caché puede pintar el asiento; solo la base decide de quién es.

Menús y pedidos

¿Y si un mismo sistema necesita dos familias?

El menú cambia de forma con cada restaurante; el pedido y el pago tienen invariantes.

ESCENARIO | **App de delivery** · Tacos Don Memo arma su menú; tú pides y pagas.

✓ EL MENÚ

Documentos
MongoDB

1 restaurante = 1 documento

```
{ "_id": "rest-311",
  "nombre": "Tacos Don Memo",
  "platillos": [
    { "nombre": "Pastor", "precio": 95,
      "tamaños": ["5 pzas", "8 pzas"],
      "extras": { "queso": 15, "piña": 10 } },
    { "nombre": "Combo gringa", "precio": 120,
      "incluye": ["gringa", "agua"] } ] }
```

✓ Una lectura trae el menú completo

+

✓ PEDIDO Y PAGO

Relacional
PostgreSQL

`pago.pedido UNIQUE`

```
BEGIN;                                          una transacción
INSERT INTO pedido (id, cliente, rest, total)
  VALUES (5001, 'U-1027', 'rest-311', 230);
INSERT INTO renglon (pedido, platillo, precio)
  VALUES (5001, 'Pastor + queso', 110),     precio
         (5001, 'Combo gringa', 120);     congelado
INSERT INTO pago (pedido, monto)
  VALUES (5001, 230);
COMMIT;
```

✓ Pedido, renglones y pago: todo o nada

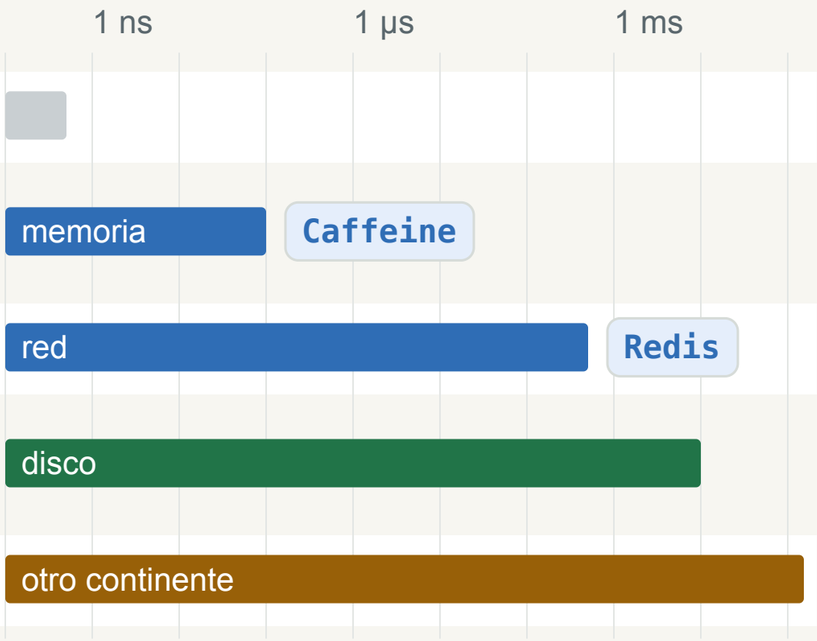
La verdad se queda donde viven los invariantes: el pedido y el pago.

Por qué una caché

¿El saldo de la app tiene que salir siempre de la base?

Una caché es una copia de un dato puesta más cerca de quien lo lee.

OPERACIÓN	LATENCIA	SI 1 NS FUERA 1 S
Referencia a caché L1	`0.5 ns`	medio segundo
Referencia a memoria principal	**`100 ns`**	**1 min 40 s**
Ida y vuelta en el mismo centro de datos	**`500 µs`**	**5.8 días**
Seek de disco	**`10 ms`**	**3.8 meses**
Paquete California, Países Bajos, California	**`150 ms`**	**4.8 años**



Cada salto, de memoria a red a disco, cuesta órdenes de magnitud.

Capacidad, no velocidad

¿Qué compra de verdad una caché?

En quincena, Mexi Banco recibe 10 000 lecturas por segundo de `GET /cuentas/{clabe}`.

Sin caché

a la base

10 000 /s

tope 5 000 /s



✗ 200 %: la cola crece sin fin

h = 90 %

a la base

1 000 /s

tope 5 000 /s



✓ 20 %

h = 99 %

a la base

100 /s

tope 5 000 /s



✓ 2 %

De 90 a 99 %: +9 puntos

✓ 10 veces menos carga

Manda la tasa de fallos:
10 % → 1 %

✗ Si Redis se cae

10 000 lecturas/s vuelven a una base que aguanta 5 000.

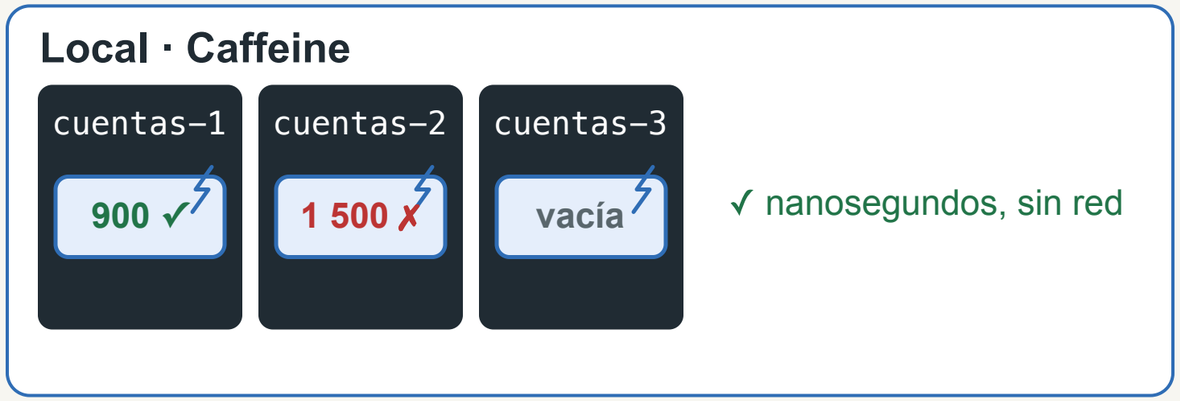
La caché compra capacidad: la base solo ve los fallos.

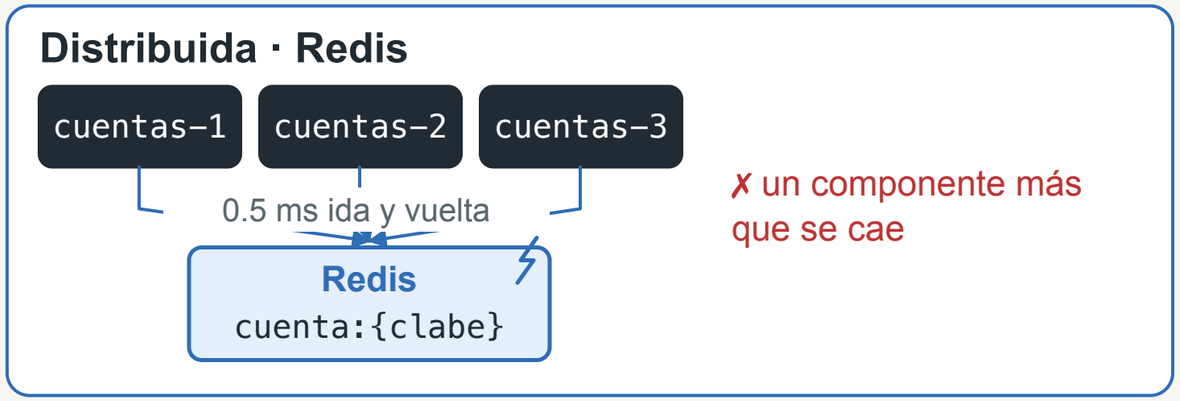
Dónde vive

¿Entre quién y quién se pone la copia?

Hay una copia posible en cada capa, y a cada una la invalida alguien distinto.







Local: N copias ciegas. Distribuida: una copia a un salto de red.

Una copia que se permite estar vieja

¿Cómo muere una entrada?

Una entrada muere de tres formas: expira, se desaloja o se invalida.

1 Expira — pasó el tiempo

```
SET cuenta:002180000000000001 '{"saldo":900}' EX 600
GET cuenta:002180000000000001   "{\"saldo\":900}"
TTL cuenta:002180000000000001   (integer) 587
```

2 Se desaloja — faltó memoria

```
maxmemory-policy allkeys-lru
```

✗ de fábrica es noeviction: rechaza escrituras

3 Se invalida — cambió la verdad ✓

```
DEL cuenta:002180000000000001   (integer) 1
GET cuenta:002180000000000001   (nil)
```

CuentaService.java · dos métodos

```java
// GET; si falla, lee la base y hace SET
@Cacheable(cacheNames = "cuenta", key = "#clabe")
public Cuenta consultar(String clabe) {
  return cuentas.findByClabe(clabe).orElseThrow();
}

// DEL cuando el método termina sin excepción
@CacheEvict(cacheNames = "cuenta", key = "#clabe")
@Transactional
public void cargar(String clabe, BigDecimal monto) {
  cuentas.cargar(clabe, monto);
}
```

El TTL de 10 min se configura aparte:
`spring.cache.redis.time-to-live`

El TTL acota cuánto dura el error; el DEL lo corrige.

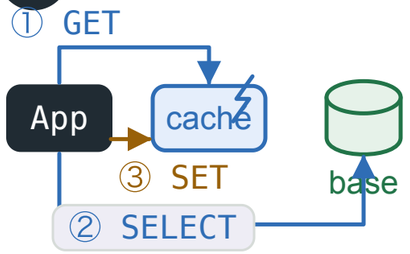
Cinco patrones con la base

¿Quién escribe en la caché y quién en la base?

Cambia quién habla con la base y cuándo se llena la copia.

AL LEER

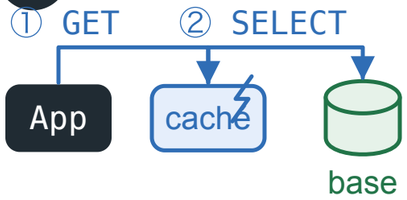
1 Cache-aside



✗ Miss caro (3 viajes) y carreras

Mexi Banco:
`GET /cuentas/{clabe}`

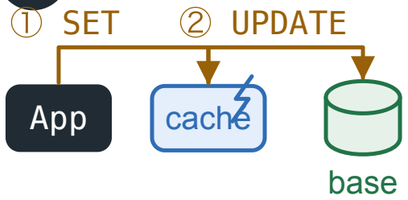
2 Read-through



✗ La caché conoce la base

AL ESCRIBIR

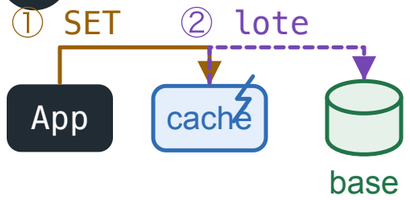
3 Write-through



síncrono

✗ Paga doble y cachea lo que nadie lee

4 Write-behind

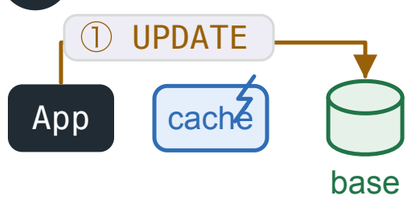


después, en lote

✗ Si la caché muere, se pierde lo ya confirmado

✗ Mexi Banco: nunca saldo ni movimientos

5 Write-around



se llena al leer

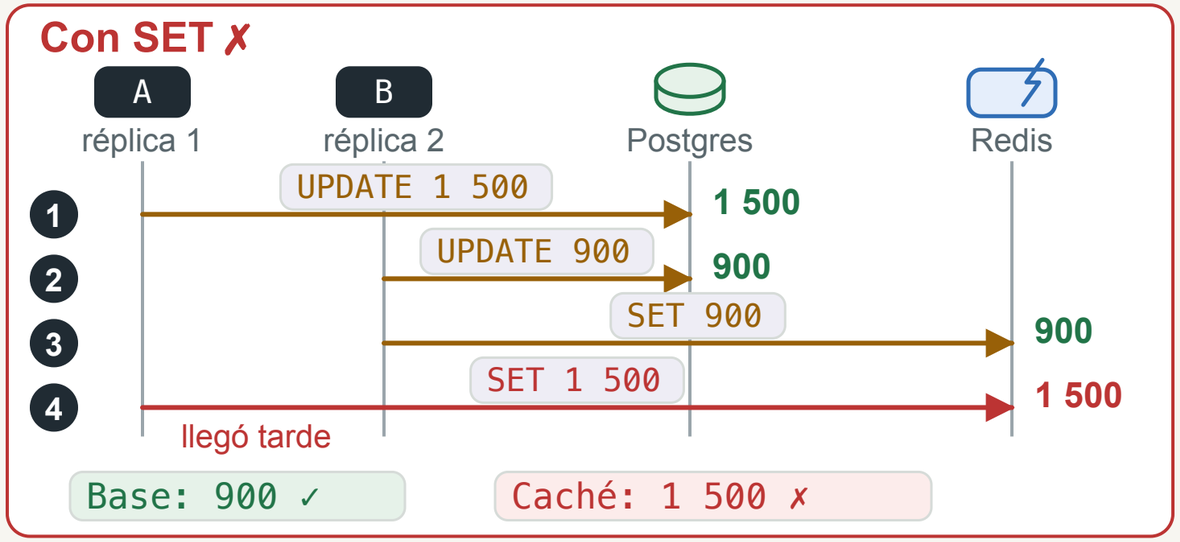
✗ Lo cacheado sigue viejo hasta TTL o DEL

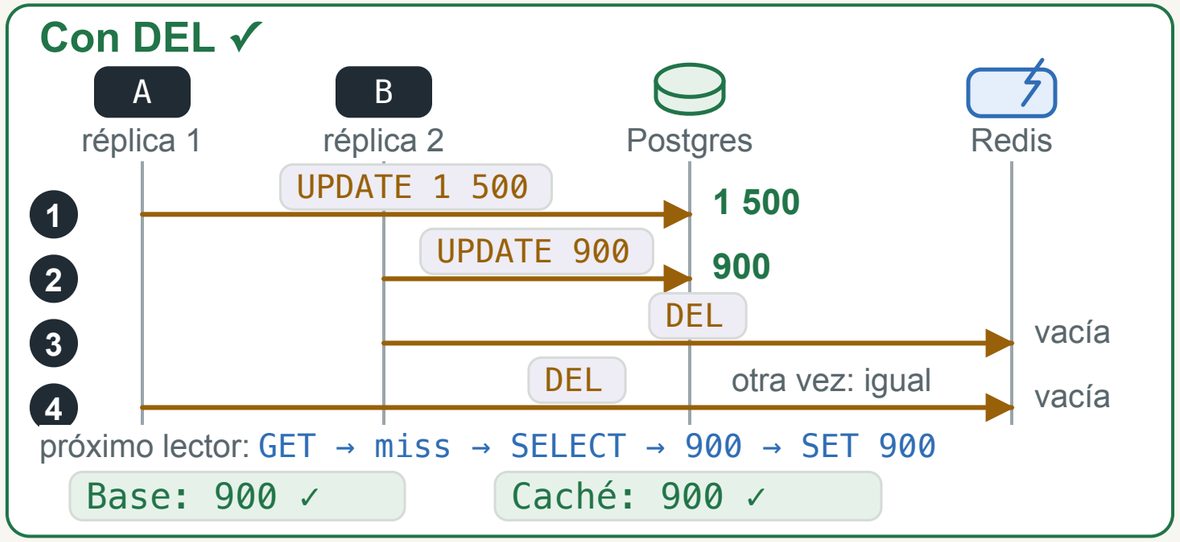
Cada patrón decide quién llena la copia y qué se pierde si cae.

Borrar, no actualizar

Al escribir, ¿actualizo la caché o la borro?

Con DEL el orden no importa: el próximo lector trae la verdad de la base.





El DEL va después del commit

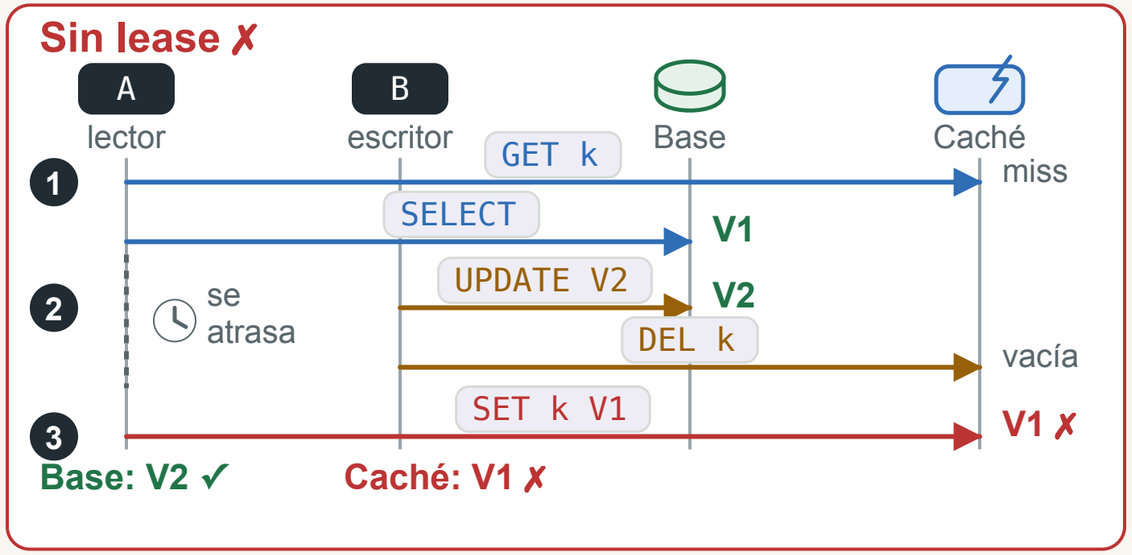
`UPDATE` → `COMMIT` → `DEL` ✓ ✗ DEL antes del COMMIT: un lector recarga el valor viejo en esa ventana

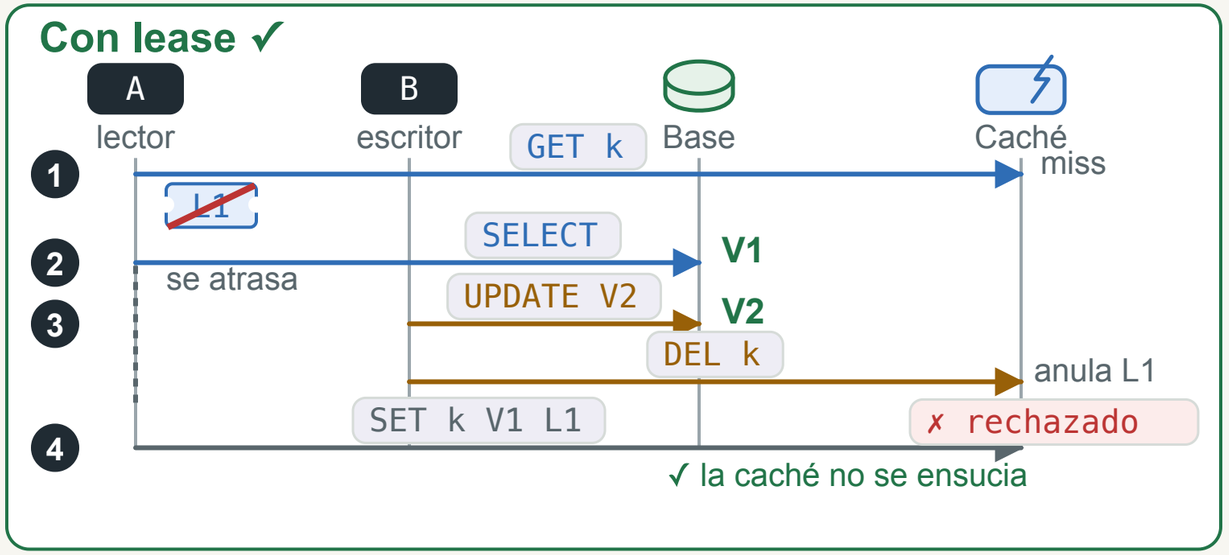
Borrar no depende del orden; actualizar sí.

Stale set y leases

¿Y si el que llena la caché llega tarde?

Un lector lento puede guardar en la caché un valor que la base ya cambió: el stale set.





Con una regla más también frena la estampida:
un lease por clave cada 10 s.

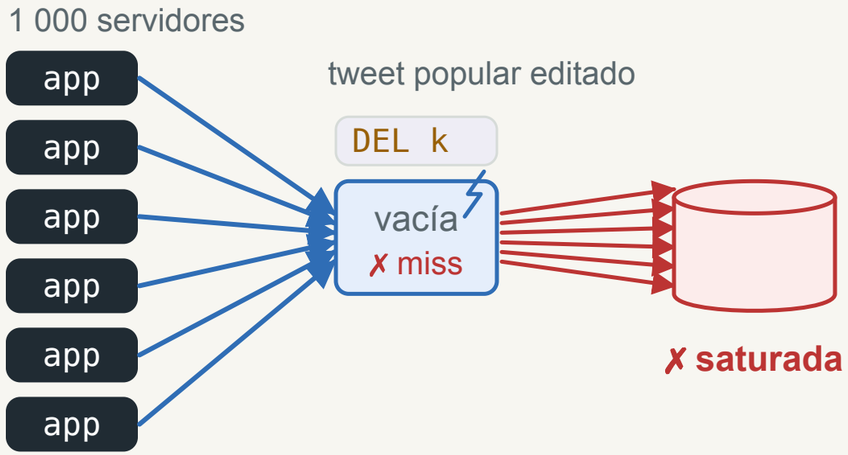


Solo llena la caché quien trae un lease que ningún DEL anuló.

Estampida

¿Qué pasa cuando miles fallan a la vez en la misma clave?

Una clave caliente expira o se invalida, y miles de lectores fallan a la vez.



1 Un solo cargador

`SET lock:k token NX PX 3000` o el lease: los demás esperan

2 Agrupar en la JVM

```
@Cacheable(cacheNames = "cuenta", sync = true)
```

3 Servir lo viejo mientras uno recarga

⚠ **Twitter · anécdota del profesor de FIUBA**

Invalidó claves de 1 % en 1 %;
una sola hot key saturó la base.

Facebook · 2010 · unas 2.5 h

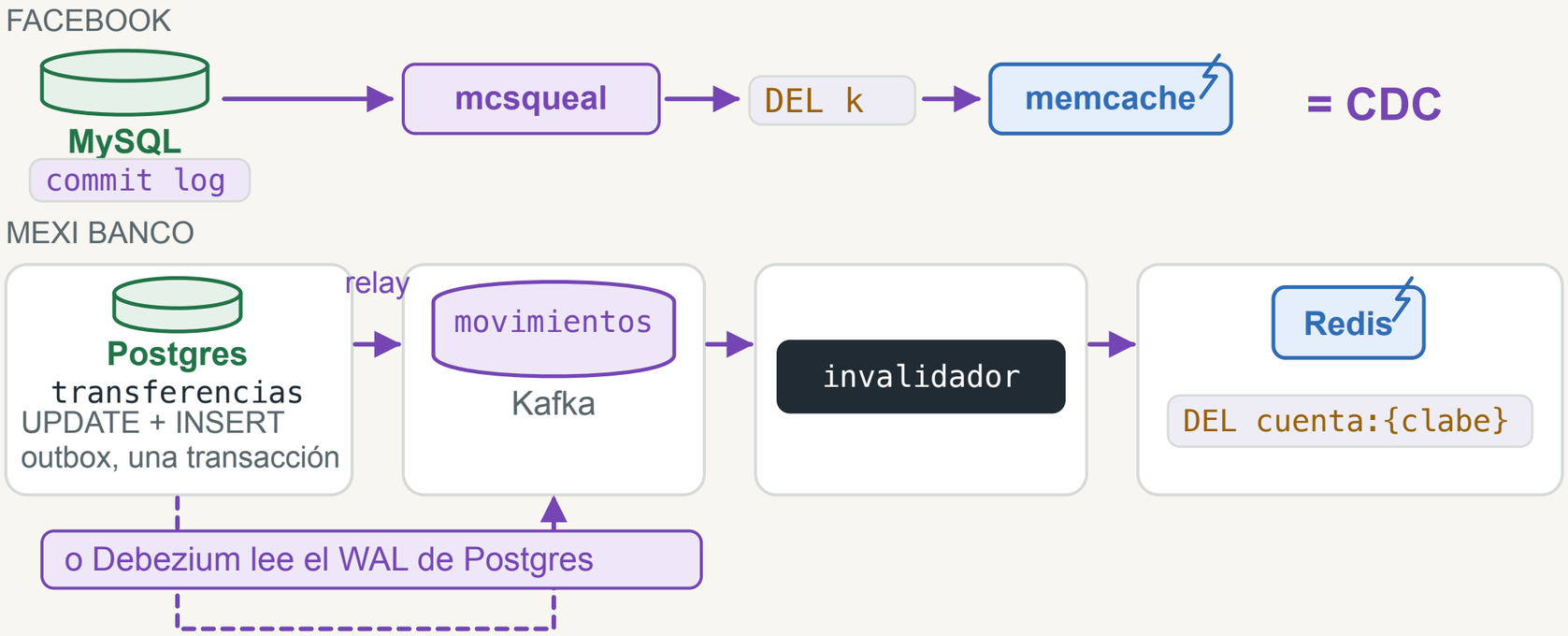
✗ Borrar al fallar la base convierte la falla en estampida.

⚠ **Una clave caliente pesa más que miles de frías.**

Invalidar con eventos

¿Y si el DEL no lo manda la app, sino el log de la base?

mcsqueal lee el commit log de la base y manda los DEL: eso es CDC.



At-least-once

el evento e7 llega dos veces

Con DEL

`e7 → DEL`
`e8 → DEL`
`e7 → DEL` (repetido)

✓ caché vacía; el lector trae 900

Con SET saldo

`e7 → SET 1 500`
`e8 → SET 900`
`e7 → SET 1 500` (repetido)

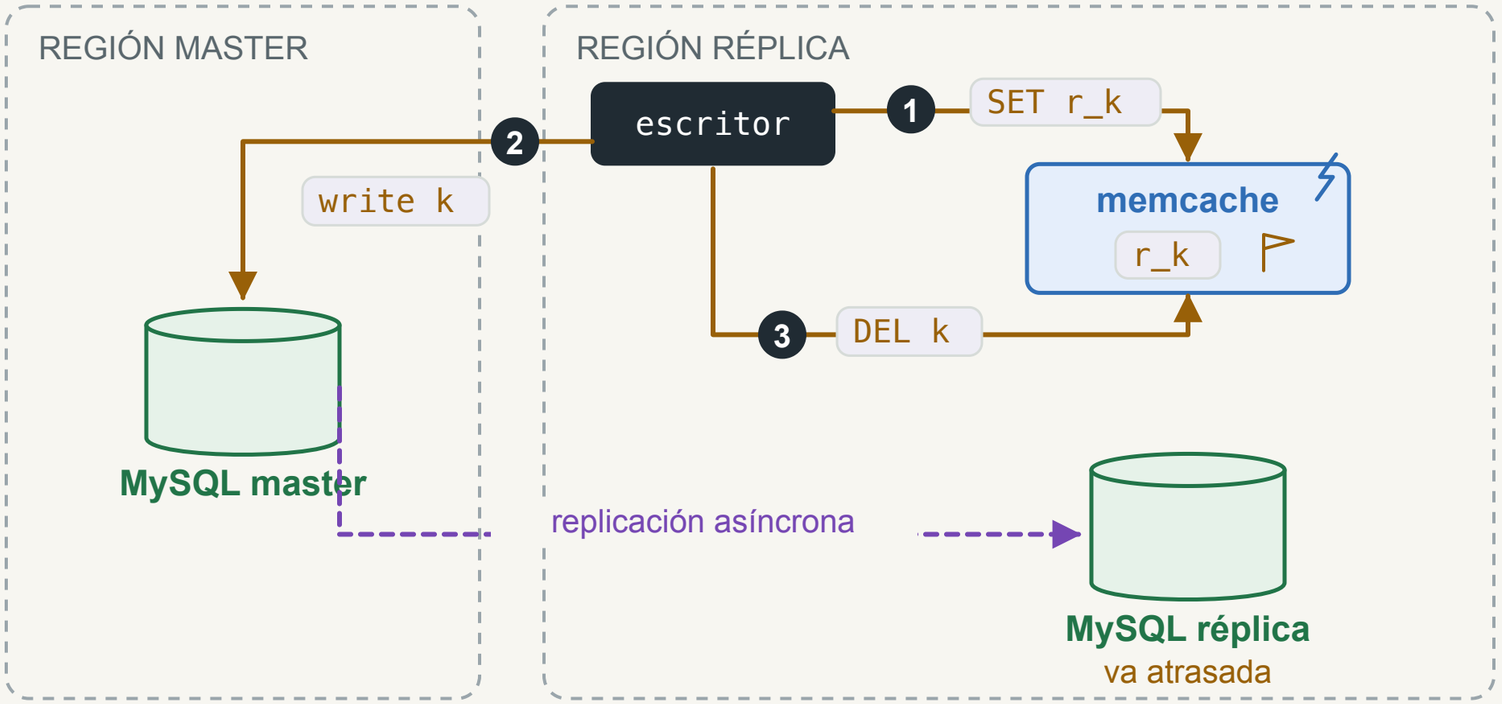
✗ caché 1 500, base 900

Que mande el DEL quien sabe qué se confirmó: el log de la base.

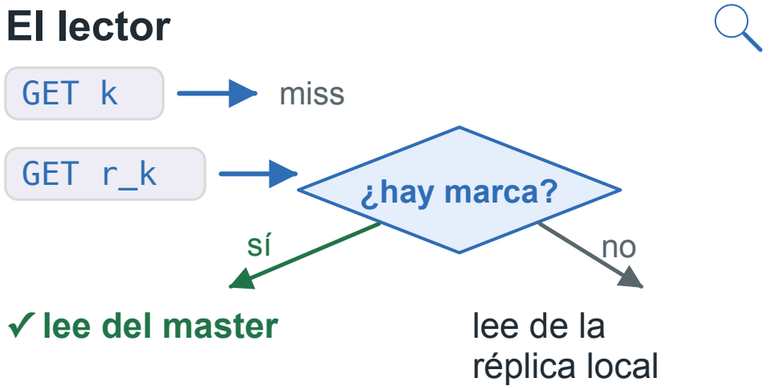
Varias regiones

¿Y si el lector rellena la caché desde una réplica atrasada?

El remote marker avisa: aquí puede estar viejo, ve al master.



El lector



mcsqueal borra r_k cuando el cambio llega a la réplica.

✗ La marca no es caché: si se desaloja, se pierde la garantía.

Una marca dice: aquí puede estar viejo, ve al master.

Qué no se cachea en Mexi Banco

¿Qué datos no pueden salir de una copia vieja?

Se cachea lo que se muestra; lo que decide dinero se lee de la verdad.

DATO	DECISIÓN
Saldo para autorizar	✗ **Nunca desde la caché** `UPDATE cuenta SET saldo = saldo - :m WHERE clabe = :c AND saldo >= :m`
Saldo para mostrar	✓ Caché con TTL corto y DEL al transferir
Catálogo de bancos SPEI	✓ Caffeine local, TTL de horas
Claves de idempotencia y sesiones	! No son caché: en la base, o Redis con `noeviction` y persistencia
Respuestas autenticadas en una CDN	✗ **Nunca:** `Cache-Control: no-store` o `private`

Steam, 2015: páginas de unos 34 000 usuarios pudieron verse por otros.

? **¿Y si no hay un dueño de la verdad que pueda invalidar las copias?**

El libro mayor

¿Dónde vive la verdad del dinero?

Una lista de movimientos en orden; el saldo se calcula leyéndola.

LIBRO MAYOR · tabla movimientos

#	cuenta	movimiento	monto
1	Ada	depósito	+1,000
2	Ada	pago a Beto	−500
3	Beto	pago de Ada	+500
4	…		

se calcula →

SALDOS

Ada $500

Beto $500

copia derivada: se puede recalcular

↓ solo se agrega al final

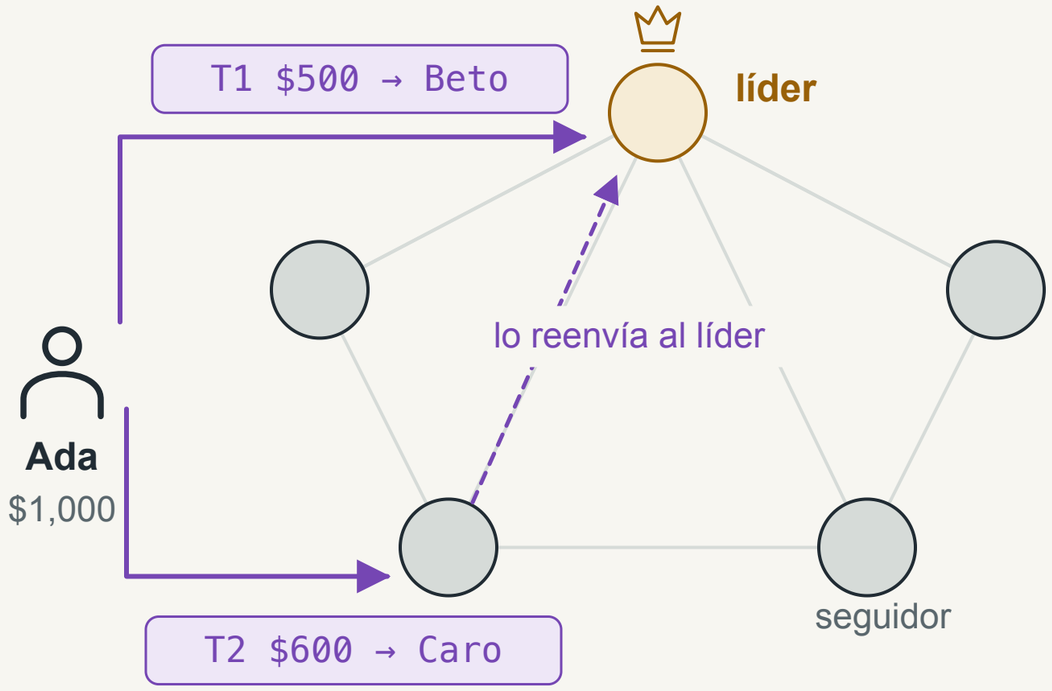
Mexi Banco: movimientos es la verdad; cuenta.saldo sale de ahí

El libro mayor es un log; el saldo es lo que resulta de leerlo.

Un libro mayor sobre Raft

¿Qué impide gastar dos veces si hay cinco copias?

El líder ordena cada pago antes de aplicarlo; el que ya no alcanza se rechaza.



LOG DEL LÍDER · se replica a la mayoría

1 · T1 Ada → Beto $500 ✓

Ada queda en $500

2 · T2 Ada → Caro $600 ✗

no alcanza: se rechaza

Raft · S06: un líder, mayoría 3 de 5

Con un líder que ordena, el doble gasto es imposible.

¿Y si nadie puede ser el dueño?

¿Por qué alguien querría un libro mayor sin banco?

31-oct-2008
el paper cumple 18 años hoy

Nakamoto (2008): pagos entre dos personas, sin un tercero de confianza.

CON UN DUEÑO



✗ Hay que confiar en él

✗ Puede revertir un pago

✗ Puede cerrarte la cuenta

quitar al dueño →

LO QUE SE PIDE

1. **Sin permisos**
   cualquiera puede entrar.

2. **Sin centro**
   nadie puede censurar un pago.

3. **A prueba de alteraciones**
   nadie reescribe la historia sin que se note.

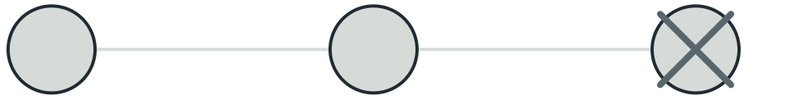
Quitar al dueño es quitar al único que ordenaba el libro.

Nadie confía en nadie

¿Qué le pasa a Raft si un nodo miente?

Falla bizantina: un nodo no solo se cae; puede mentir.

LO QUE RAFT ASUME

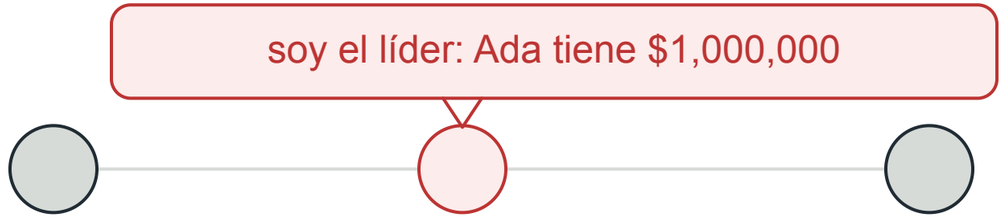


caído: se calla

✓ Tolera 2 caídos de 5

✓ Todas las máquinas son mías

SIN DUEÑO



✗ Puede mentir, no solo caerse

✗ Nadie sabe cuántos nodos hay

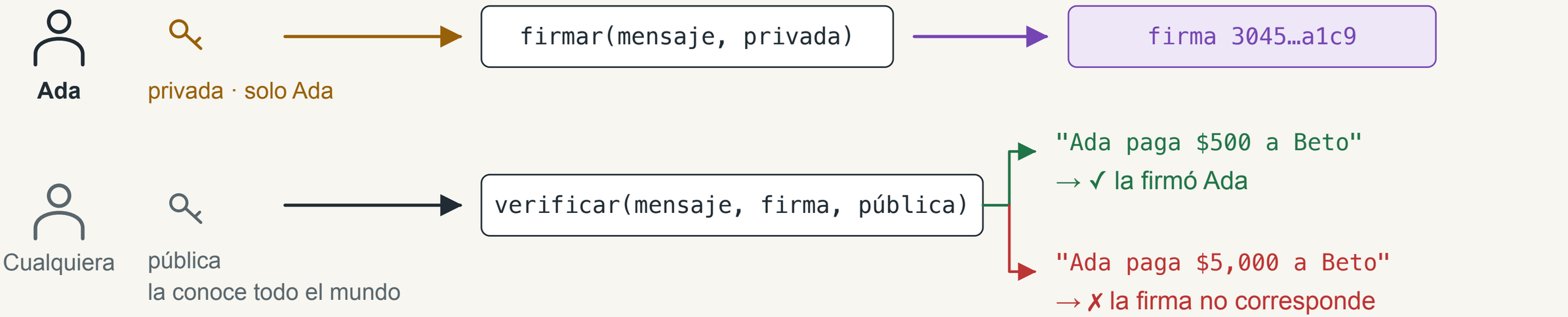
Generales bizantinos · Lamport, Shostak y Pease (1982)

Sin dueño hay que suponer que algunos nodos mienten.

Identidad sin login

¿Cómo sé que el pago lo autorizó Ada?

Cada quien tiene un par de llaves: firma con la privada y cualquiera verifica con la pública.



`de: llave pública de Ada · para: llave pública de Beto · $500 · firma 3045…a1c9`

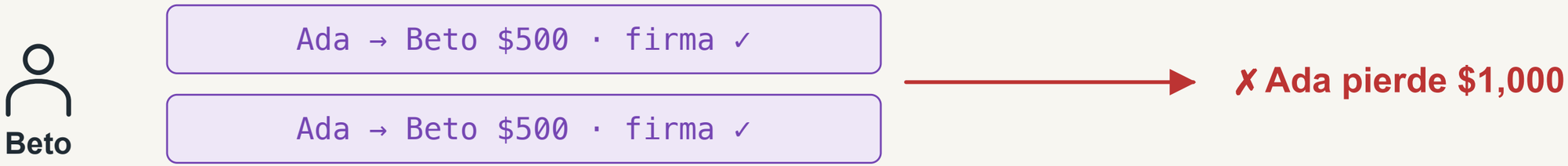
La llave pública es tu cuenta; la privada, tu firma.

Repetir una firma

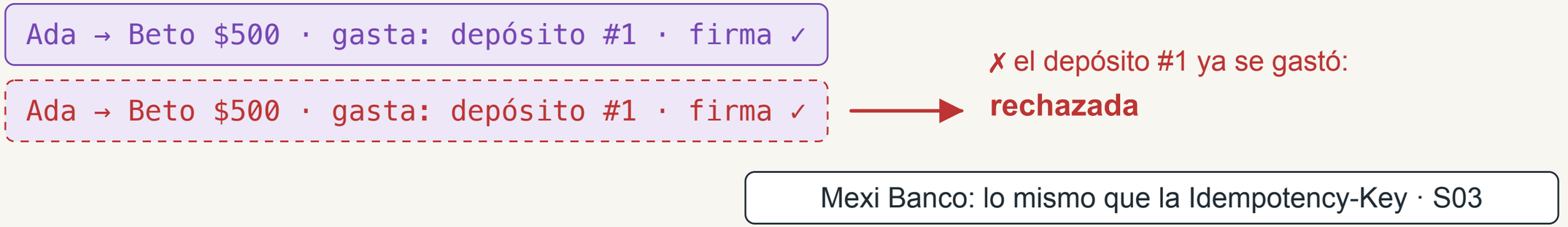
¿Qué impide que Beto cobre el mismo pago dos veces?

Una firma válida se puede copiar; por eso cada pago dice qué dinero gasta.

SIN REFERENCIA



CON REFERENCIA



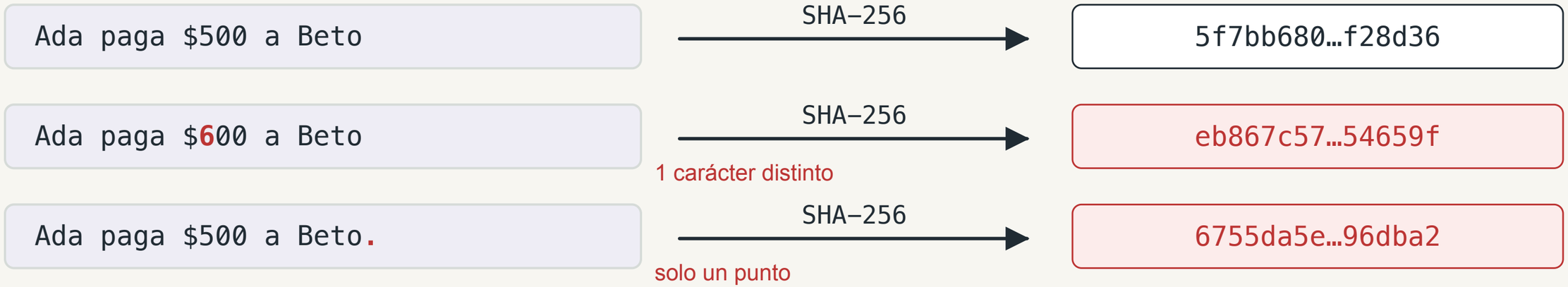
Mexi Banco: lo mismo que la Idempotency-Key · S03

Cada pago apunta a lo que gasta; gastarlo dos veces se nota.

La huella digital

¿Cómo noto que alguien cambió un dato?

Un hash convierte cualquier dato en una huella fija; SHA-256 da 64 caracteres.



1. **Mismo dato, misma huella**
2. **Un cambio mínimo, otra huella**
3. **De la huella no se regresa al dato**

El hash no impide cambiar el dato; delata que cambió.

Bloques encadenados

¿Qué pasa si alguien cambia un pago viejo?

Los pagos se agrupan en bloques, y cada bloque guarda el hash del anterior.



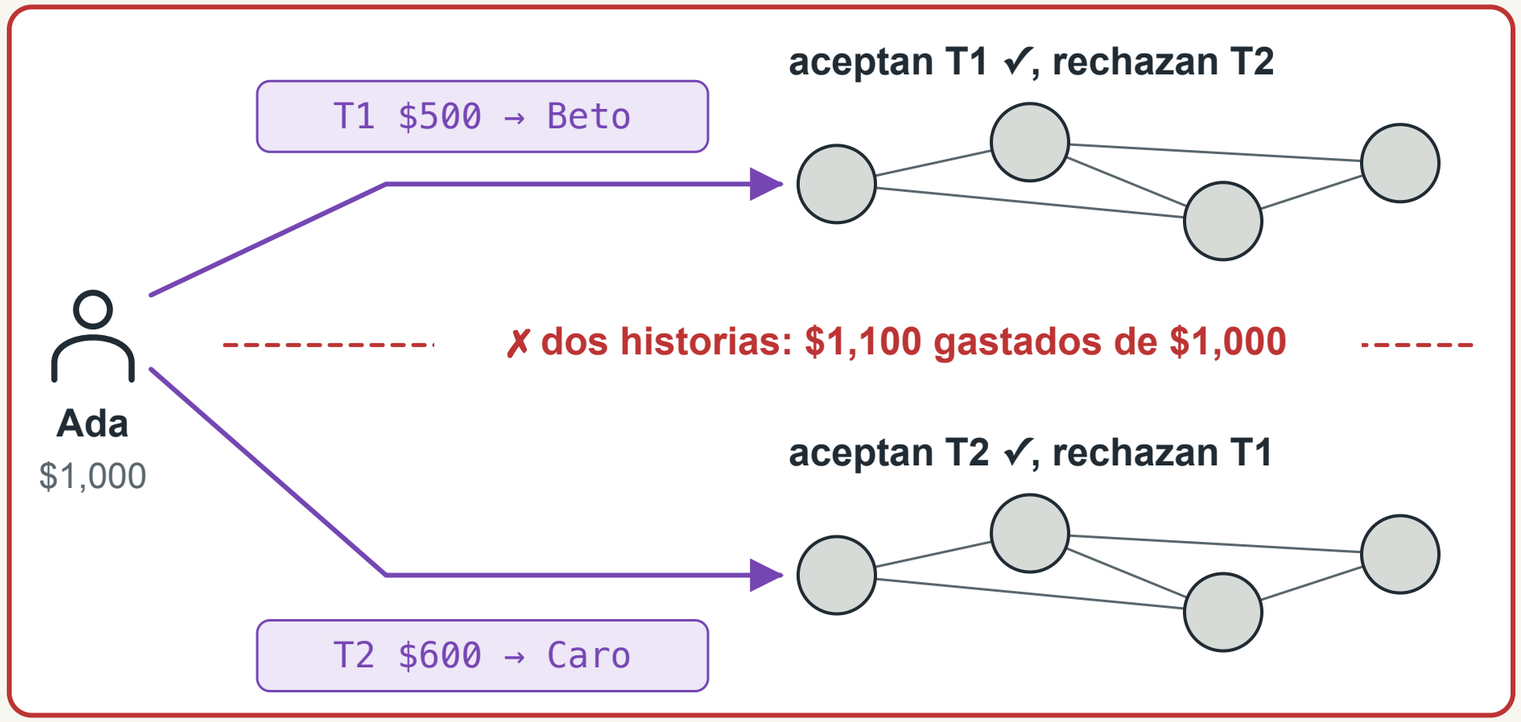
Es un log: cada bloque es posterior al anterior · S10

Alterar un bloque viejo se nota: rompe a todos los que siguen.

Sin líder vuelve el doble gasto

¿Qué pasa si nadie ordena los pagos?

Cada nodo pasa lo que recibe a sus vecinos (gossip) y acepta el primer pago que le llega.



LO QUE YA TENEMOS

✓ Quién firmó: llaves

✓ Si alguien alteró: hash

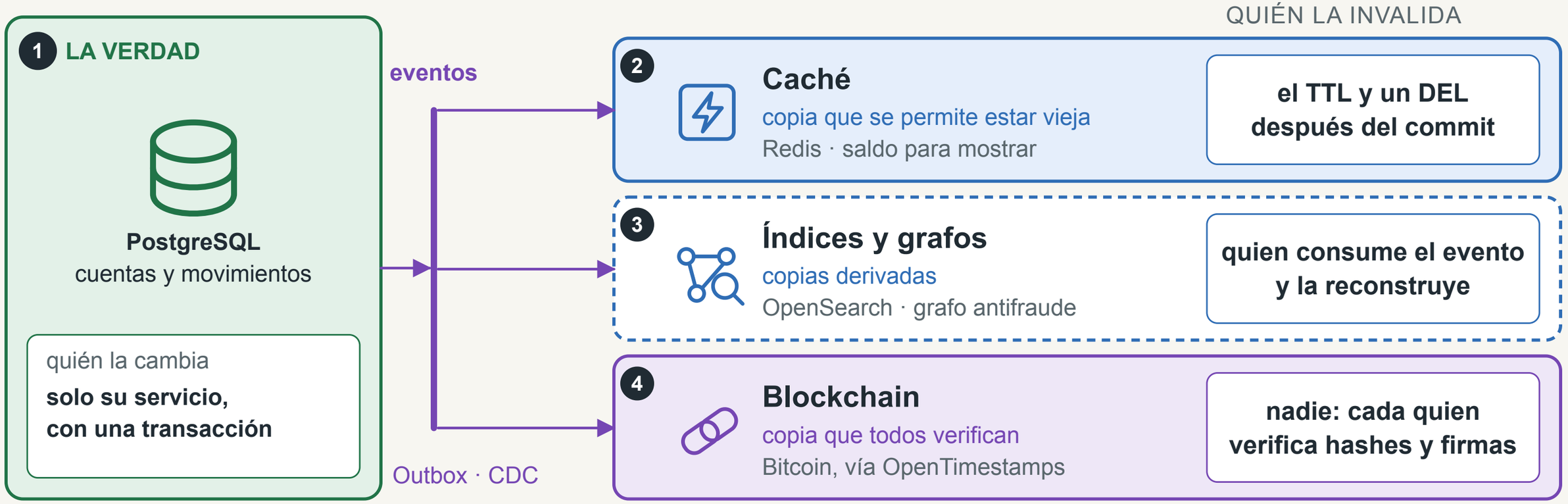
✗ **Cuál fue primero: orden**

Continúa en la S12

Falta el orden: ¿quién ordena si nadie es el dueño?

Una verdad, varias copias

¿Dónde vive cada dato, y qué pasa cuando vive en más de un lugar?



Por cada copia, decide quién la invalida.